

Intrusive and Non-Intrusive Model Reduction

Homework 0 – uv, JAX, and the EPFL cluster

Ungraded setup sheet

Exercise 0.1: uv and a first JAX script

Install the Python package manager `uv` on your laptop (<https://docs.astral.sh/uv/>).

- Create a new project with `uv init mor26-sheet0`. Add JAX with `uv add jax`.
- Write a script `hello_jax.py` that prints `jax.devices()`, samples a matrix $A \in \mathbb{R}^{4 \times 4}$ with independent standard normally distributed entries, and prints $A^T A$. Run it with `uv run hello_jax.py`. Which hardware does JAX use by default?

Exercise 0.2: Row swaps: NumPy versus JAX

Consider the following Python script, which exchanges the rows i and j of a matrix A , as needed for, e.g., partial pivoting.

```
import numpy as np

def swap_rows(A, i, j):
    A[[i, j]] = A[[j, i]]
    return A
```

- Apply the function to a matrix $A \in \mathbb{R}^{4 \times 4}$. Describe its effect on the argument.
- Apply the function to the same matrix as a `jax.numpy (jnp)` array. What happens?
- Write a function `swap_rows_jax(A, i, j)` that is JAX compatible and returns a matrix with the rows i and j exchanged.

Exercise 0.3: Gradients of a quadratic function

Let $A = A^T \in \mathbb{R}^{n \times n}$, let $b \in \mathbb{R}^n$, and let

$$f(x) := \frac{1}{2} x^T A x - b^T x.$$

- Write a JAX compatible function `f(A, b, x)` that evaluates $f(x)$.
- Compute $\nabla f(x)$ by hand and write a function `grad_f(A, b, x)` that returns $\nabla f(x)$.
- Write a function `jax_grad_f(A, b, x)` that returns the gradient $\nabla f(x)$ using `jax.grad`.

Exercise 0.4: Parametric solves, snapshots, and timings

Let $A_0, A_1 \in \mathbb{R}^{n \times n}$, let $b \in \mathbb{R}^n$, and, for a parameter $\mu \in \mathbb{R}$, let

$$A(\mu) := A_0 + \mu A_1, \quad u(\mu) := A(\mu)^{-1} b.$$

- Write a function `solve(mu, A0, A1, b)` that returns $u(\mu)$, using `jnp.linalg.solve`.
- Write a function `snapshots(mus, A0, A1, b)` that returns, for parameters $\mu_1, \dots, \mu_k$ given as a vector `mus`, the snapshot matrix

$$U := [u(\mu_1) \ \cdots \ u(\mu_k)] \in \mathbb{R}^{n \times k}$$

with a call of `jax.vmap`. Do not loop over the parameters.

- Generate random test data with $n = 1024$, $k = 16$, $A_0 = nI + R + R^T$ with standard normally distributed R , A_1 , and b , and equidistant $\mu_1, \dots, \mu_k \in [0, 1]$. Use the test data to compare the run times of a Python loop over `solve`, of `snapshots`, and of `jax.jit(snapshots)`, timing the first call of the compiled function separately and using `jax.block_until_ready`. Which variant is the fastest?

- (d) Open <https://colab.research.google.com/>, switch to a GPU runtime, and check with `jax.devices()` that JAX uses the GPU. Repeat (c) for increasing n . For which problem sizes does the GPU pay off?
- (e) Compute the singular value decomposition $U = W\Sigma V^T$ of the snapshot matrix with `jnp.linalg.svd` and plot the singular values $\sigma_1 \geq \dots \geq \sigma_k$ on a semi-logarithmic scale. Determine the smallest r such that the best rank- r approximation U_r of U satisfies

$$\|U - U_r\|_F \leq 10^{-5} \|U\|_F.$$

Use `jnp.einsum` to compute the Frobenius norm.

Exercise 0.5: The EPFL cluster: SCITAS and Izar

- (a) Ask A. Karsai to include you in the SCITAS user group to gain access to the EPFL HPC cluster.
- (b) Test the connection using

```
ssh <username>@izar.hpc.epfl.ch
```

If successful, `exit` the cluster and configure a passwordless ssh connection using

```
ssh-keygen -t ed25519 -C "epfl" -f ${HOME}/.ssh/epfl
ssh-copy-id -i ${HOME}/.ssh/epfl.pub <username>@izar.hpc.epfl.ch
```

Save this identity in the ssh config file using

```
cat >> ~/.ssh/config <<'EOF'
### EPFL SCITAS (Izar)
Host epfl-izar
    HostName izar.hpc.epfl.ch
    User <username>
    IdentityFile ~/.ssh/epfl
    IdentitiesOnly yes
EOF
```

and verify that everything works by connecting using `ssh epfl-izar`.

- (c) (optional) Edit code on the cluster with VS Code, following the SCITAS guide on using VS Code.
- (d) Copy your uv project to the cluster using `scp` or `git`. On the login node, install `uv` and add the CUDA variant of JAX with `uv add "jax[cuda12]"` (Izar's V100 GPUs are not supported by JAX's CUDA 13 wheels).
- (e) Write a script `hw0_gpu.py` that prints `jax.devices()`, runs `jax.jit(snapshots)` from Exercise 0.4 with $n = 4096$ and $k = 16$ and prints the run times, and raises an error if no GPU is found.
- (f) Submit the script as a Slurm job with the following batch script `hw0.sbatch` (`uv` installs into `~.local/bin`, hence the absolute path). Find out in the SCITAS documentation what each directive requests:

```
#!/bin/bash
#SBATCH --job-name=hw0
#SBATCH --partition=gpu
#SBATCH --qos=debug
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --gres=gpu:1
#SBATCH --time=00:10:00
#SBATCH --output=hw0.out

$HOME/.local/bin/uv run python hw0_gpu.py
```

Submit the job with `sbatch hw0.sbatch`. Monitor it with `squeue`. Inspect the job's output file: the output has to show that the GPU was used.